FUNDAÇÃO DE ASSISTÊNCIA E EDUCAÇÃO
CENTRO UNIVERSITÁRIO ESPÍRITO-SANTENSE
CURSO DE GRADUAÇÃO EM PYTHON

CLEITON LEONEL CRETON

PROGRAMAÇÃO ORIENTADA A OBJETOS

CARIACICA

2025

CLEITON LEONEL CRETON

PROGRAMAÇÃO ORIENTADA A OBJETOS

Trabalho acadêmico do Curso de Graduação em Python apresentado ao Centro universitário espírito-santense como parte das exigências da Disciplina Programação orientada a objetos, sob orientação do(a) Professor(a) Cleiton L. Creton.

CARIACICA

2025

SUMÁRIO

INTRODUÇÃO.....4

DESENVOLVIMENTO.....5

CONCLUSÃO.....8

REFERÊNCIAS.....9

INTRODUÇÃO

A Programação Orientada a Objetos (POO) é um paradigma de programação que utiliza objetos para estruturar um programa de software. Em vez de escrever código procedural que realiza tarefas sequencialmente, a POO organiza o código em torno de objetos, que combinam dados (atributos) e ações (métodos). Python é uma linguagem de programação que suporta POO, permitindo que os desenvolvedores criem aplicativos modulares, reutilizáveis e fáceis de manter.

Este trabalho explora os principais conceitos da POO em Python, incluindo classes, objetos, herança, polimorfismo e encapsulamento. Serão apresentados exemplos práticos de como aplicar esses conceitos para resolver problemas de programação. A POO facilita o desenvolvimento de sistemas complexos, promovendo a reutilização de código e a organização modular.

DESENVOLVIMENTO

Classes e Objetos

Uma classe é um modelo para criar objetos. Define os atributos (dados) e métodos (ações) que os objetos terão. Um objeto é uma instância de uma classe. Por exemplo, podemos definir uma classe chamada 'Carro' com atributos como 'cor', 'modelo' e métodos como 'ligar', 'acelerar'.

Exemplo de definição de uma classe em Python:

```python
class Carro:
 def __init__(self, cor, modelo):
 self.cor = cor
 self.modelo = modelo
 self.velocidade = 0

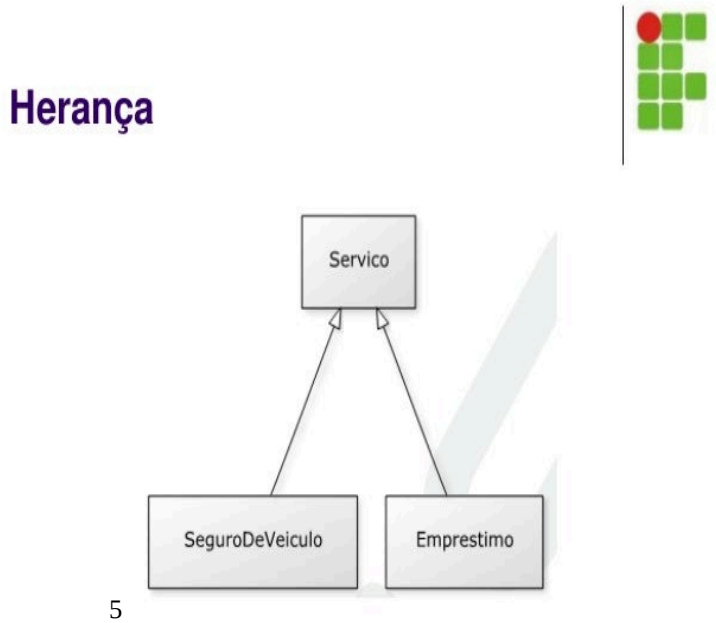
 def ligar(self):
 print("Carro ligado!")

 def acelerar(self, incremento):
 self.velocidade += incremento
 print(f"Velocidade atual: {self.velocidade} km/h")
```

Neste exemplo, `__init__` é o construtor da classe, que inicializa os atributos do objeto quando ele é criado. Os métodos `ligar` e `acelerar` definem as ações que o objeto pode realizar.

Herança

Diagrama ilustrando o conceito de herança em POO, onde classes filhas herdam atributos e métodos da classe pai.



Herança permite que uma classe (classe filha) herde atributos e métodos de outra classe (classe pai). Isso promove a reutilização de código e a criação de hierarquias de classes. Por exemplo, podemos criar uma classe `CarroEletrico` que herda da classe `Carro` e adiciona atributos e métodos específicos para carros elétricos.

Exemplo de herança em Python:

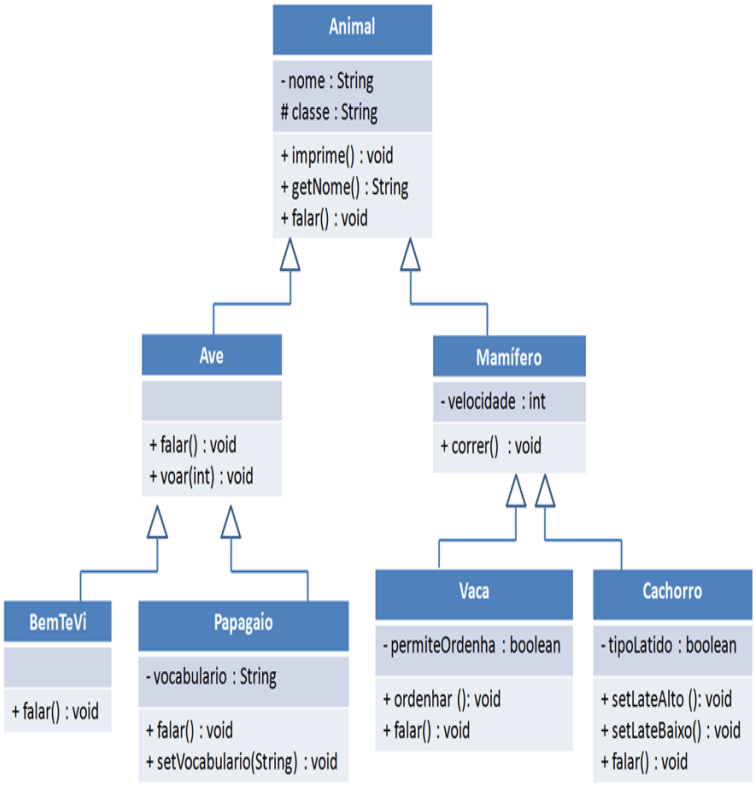
```python
class CarroEletrico(Carro):
 def __init__(self, cor, modelo, capacidade_bateria):
 super().__init__(cor, modelo)
 self.capacidade_bateria = capacidade_bateria

 def carregar(self):
 print("Carro elétrico carregando...")
```

Neste exemplo, `CarroEletrico` herda os atributos `cor` e `modelo` da classe `Carro` e adiciona o atributo `capacidade_bateria` e o método `carregar`.

Polimorfismo

Polimorfismo permite que objetos de diferentes classes sejam tratados de forma uniforme. Isso significa que um método pode ter comportamentos diferentes dependendo da classe do objeto que o chama. Por exemplo, podemos ter um método `descrever` que se comporta de maneira diferente para objetos das classes `Carro` e `Bicicleta`.



Representação visual do polimorfismo, mostrando como diferentes objetos podem responder ao mesmo método de maneiras distintas.

Exemplo de polimorfismo em Python:

```
```python
def descrever(veiculo):
 print(f"Este é um veículo do tipo {type(veiculo).__name__}")

carro = Carro("vermelho", "Sedan")
bicicleta = Bicicleta("azul", "Mountain Bike")

descrever(carro)
descrever(bicicleta)
```
```

Encapsulamento

Encapsulamento é o princípio de ocultar os detalhes internos de um objeto e fornecer uma interface pública para interagir com ele. Isso protege os dados do objeto de acesso não autorizado e permite que a implementação interna seja alterada sem afetar o código que usa o objeto.

CONCLUSÃO

A Programação Orientada a Objetos em Python oferece uma maneira poderosa e flexível de estruturar e organizar o código. Os conceitos de classes, objetos, herança, polimorfismo e encapsulamento permitem que os desenvolvedores criem aplicativos modulares, reutilizáveis e fáceis de manter. Ao dominar esses conceitos, os programadores podem desenvolver soluções de software mais eficientes e escaláveis.

REFERÊNCIAS

GAMMA, Erich; HELM, Richard; JOHNSON, Ralph; VLISSIDES, John. Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley, 1994.

BEAZLEY, David M. Python Essential Reference. 4th ed. Addison-Wesley Professional, 2009.

LUTZ, Mark. Learning Python. 5th ed. O'Reilly Media, 2013.

SUMMERFIELD, Mark. Programming in Python 3: A Complete Introduction to the Python Language. 2nd ed. Addison-Wesley Professional, 2009.

SHAW, Zed A. Learn Python 3 the Hard Way: A Very Simple Introduction to the Terrifyingly Beautiful World of Computers and Code. Addison-Wesley, 2017.